

The parameter weekday is True if it is a weekday, and the parameter vacation is True if we are on vacation. We sleep in if it is not a weekday or we're on vacation. Return True if we sleep in.

sleep_in(False, False) → True
sleep_in(True, False) → False
sleep_in(False, True) → True

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def sleep_in(weekday, vacation):  
    if not weekday or vacation:  
        return True  
    else:  
        return False
```

Expected	Run		
sleep_in(False, False) → True	True	OK	
sleep_in(True, False) → False	False	OK	
sleep_in(False, True) → True	True	OK	
sleep_in(True, True) → True	True	OK	

 All Correct

[next](#) | [chance](#)

Python > Warmup-1

[done page](#)

Code is saved so long as this session is active. Create an account above to save code past this session.

Your [progress graph](#) for this problem

Warmup-1 > diff21

[prev](#) | [next](#) | [chance](#)

Given an int n, return the absolute difference between n and 21, except return double the absolute difference if n is over 21.

diff21(19) → 2
diff21(10) → 11
diff21(21) → 0

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def diff21(n):  
    if n <= 21:  
        return 21 - n  
    else:  
        return (n - 21) * 2
```

Expected	Run	
diff21(19) → 2	2	OK
diff21(10) → 11	11	OK
diff21(21) → 0	0	OK
diff21(22) → 2	2	OK
diff21(25) → 8	8	OK
diff21(30) → 18	18	OK
diff21(0) → 21	21	OK
diff21(1) → 20	20	OK
diff21(2) → 19	19	OK
diff21(-1) → 22	22	OK
diff21(-2) → 23	23	OK
diff21(50) → 58	58	OK



All Correct

Warmup-1 > parrot_trouble

[prev](#) | [next](#) | [chance](#)

We have a loud talking parrot. The "hour" parameter is the current hour time in the range 0..23. We are in trouble if the parrot is talking and the hour is before 7 or after 20. Return True if we are in trouble.

parrot_trouble(True, 6) → True
parrot_trouble(True, 7) → False
parrot_trouble(False, 6) → False

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def parrot_trouble(talking, hour):  
    return talking and (hour < 7 or hour > 20)
```

Expected	Run		
parrot_trouble(True, 6) → True	True	OK	
parrot_trouble(True, 7) → False	False	OK	
parrot_trouble(False, 6) → False	False	OK	
parrot_trouble(True, 21) → True	True	OK	
parrot_trouble(False, 21) → False	False	OK	
parrot_trouble(False, 20) → False	False	OK	
parrot_trouble(True, 23) → True	True	OK	
parrot_trouble(False, 23) → False	False	OK	
parrot_trouble(True, 20) → False	False	OK	
parrot_trouble(False, 12) → False	False	OK	

 All Correct

Warmup-1 > makes10

[prev](#) | [next](#) | [chance](#)

Given 2 ints, a and b, return True if one if them is 10 or if their sum is 10.

makes10(9, 10) → True
makes10(9, 9) → False
makes10(1, 9) → True

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def makes10(a, b):  
    return (a == 10 or b == 10) or (a + b == 10)
```

Expected	Run		
makes10(9, 10) → True	True	OK	
makes10(9, 9) → False	False	OK	
makes10(1, 9) → True	True	OK	
makes10(10, 1) → True	True	OK	
makes10(10, 10) → True	True	OK	
makes10(8, 2) → True	True	OK	
makes10(8, 3) → False	False	OK	
makes10(10, 42) → True	True	OK	
makes10(12, -2) → True	True	OK	

 All Correct

[next](#) | [chance](#)

Given an int n, return True if it is within 10 of 100 or 200. Note: abs(num) computes the absolute value of a number.

near_hundred(93) → True
near_hundred(90) → True
near_hundred(89) → False

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def near_hundred(n):  
    return (abs(100 - n) <= 10) or (abs(200 - n) <= 10)
```

Go

Editor font size %: 100 ▼

Shorter output ☐

Expected

Run

near_hundred(93) → True	True	OK	
near_hundred(90) → True	True	OK	
near_hundred(89) → False	False	OK	
near_hundred(110) → True	True	OK	
near_hundred(111) → False	False	OK	
near_hundred(121) → False	False	OK	
near_hundred(-101) → False	False	OK	
near_hundred(-209) → False	False	OK	
near_hundred(190) → True	True	OK	
near_hundred(209) → True	True	OK	
near_hundred(0) → False	False	OK	
near_hundred(5) → False	False	OK	
near_hundred(-50) → False	False	OK	
near_hundred(191) → True	True	OK	
near_hundred(189) → False	False	OK	
near_hundred(200) → True	True	OK	
near_hundred(210) → True	True	OK	
near_hundred(211) → False	False	OK	
near_hundred(290) → False	False	OK	



All Correct

Given 2 int values, return True if one is negative and one is positive. Except if the parameter "negative" is True, then return True only if both are negative.

pos_neg(1, -1, False) → True
pos_neg(-1, 1, False) → True
pos_neg(-4, -5, True) → True

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def pos_neg(a, b, negative):  
    if negative:  
        return (a < 0 and b < 0)  
    else:  
        return (a < 0 and b > 0) or (a > 0 and b < 0)
```

Go

Editor font size %: 100 ▾

Shorter output ☐

Expected

Run

pos_neg(1, -1, False) → True	True	OK	
pos_neg(-1, 1, False) → True	True	OK	
pos_neg(-4, -5, True) → True	True	OK	
pos_neg(-4, -5, False) → False	False	OK	
pos_neg(-4, 5, False) → True	True	OK	
pos_neg(-4, 5, True) → False	False	OK	
pos_neg(1, 1, False) → False	False	OK	
pos_neg(-1, -1, False) → False	False	OK	
pos_neg(1, -1, True) → False	False	OK	
pos_neg(-1, 1, True) → False	False	OK	
pos_neg(1, 1, True) → False	False	OK	
pos_neg(-1, -1, True) → True	True	OK	
pos_neg(5, -5, False) → True	True	OK	
pos_neg(-6, 6, False) → True	True	OK	
pos_neg(-5, -6, False) → False	False	OK	
pos_neg(-2, -1, False) → False	False	OK	
pos_neg(1, 2, False) → False	False	OK	
pos_neg(-5, 6, True) → False	False	OK	
pos_neg(-5, -5, True) → True	True	OK	



All Correct

Given a string, return a new string where "not " has been added to the front. However, if the string already begins with "not", return the string unchanged.

not_string('candy') → 'not candy'
not_string('x') → 'not x'
not_string('not bad') → 'not bad'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def not_string(str):  
    if len(str) >= 3 and str[:3] == "not":  
        return str  
    return "not " + str
```

Expected	Run		
not_string('candy') → 'not candy'	'not candy'	OK	
not_string('x') → 'not x'	'not x'	OK	
not_string('not bad') → 'not bad'	'not bad'	OK	
not_string('bad') → 'not bad'	'not bad'	OK	
not_string('not') → 'not'	'not'	OK	
not_string('is not') → 'not is not'	'not is not'	OK	
not_string('no') → 'not no'	'not no'	OK	

 All Correct

Given a non-empty string and an int n, return a new string where the char at index n has been removed. The value of n will be a valid index of a char in the original string (i.e. n will be in the range 0..len(str)-1 inclusive).

missing_char('kitten', 1) → 'ktten'
missing_char('kitten', 0) → 'itten'
missing_char('kitten', 4) → 'kittn'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def missing_char(str, n):  
    front = str[:n]  
    back = str[n+1:]  
    return front + back
```

Expected	Run		
missing_char('kitten', 1) → 'ktten'	'ktten'	OK	
missing_char('kitten', 0) → 'itten'	'itten'	OK	
missing_char('kitten', 4) → 'kittn'	'kittn'	OK	
missing_char('Hi', 0) → 'i'	'i'	OK	
missing_char('Hi', 1) → 'H'	'H'	OK	
missing_char('code', 0) → 'ode'	'ode'	OK	
missing_char('code', 1) → 'cde'	'cde'	OK	
missing_char('code', 2) → 'coe'	'coe'	OK	
missing_char('code', 3) → 'cod'	'cod'	OK	
missing_char('chocolate', 8) → 'chocolat'	'chocolat'	OK	

 All Correct

Given a string, return a new string where the first and last chars have been exchanged.

front_back('code') → 'eodc'
front_back('a') → 'a'
front_back('ab') → 'ba'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def front_back(str):  
    if len(str) <= 1:  
        return str  
  
    mid = str[1:-1] # символы между первым и последним  
  
    # последний символ + середина + первый символ  
    return str[-1] + mid + str[0]
```

Expected	Run	
front_back('code') → 'eodc'	'eodc'	OK
front_back('a') → 'a'	'a'	OK
front_back('ab') → 'ba'	'ba'	OK
front_back('abc') → 'cba'	'cba'	OK
front_back('') → ''	''	OK
front_back('Chocolate') → 'ehocolatC'	'ehocolatC'	OK
front_back('aavJ') → 'Java'	'Java'	OK
front_back('hello') → 'oellh'	'oellh'	OK



All Correct

next | change

Given two int values, return their sum. Unless the two values are the same, then return double their sum.

sum_double(1, 2) → 3
sum_double(3, 2) → 5
sum_double(2, 2) → 8

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def sum_double(a, b):  
    if a == b:  
        return (a + b) * 2  
    else:  
        return a + b
```

Expected	Run		
sum_double(1, 2) → 3	3	OK	
sum_double(3, 2) → 5	5	OK	
sum_double(2, 2) → 8	8	OK	
sum_double(-1, 0) → -1	-1	OK	
sum_double(3, 3) → 12	12	OK	
sum_double(0, 0) → 0	0	OK	
sum_double(0, 1) → 1	1	OK	
sum_double(3, 4) → 7	7	OK	



All Correct

[next](#) | [chance](#)

Given a string, we'll say that the front is the first 3 chars of the string. If the string length is less than 3, the front is whatever is there. Return a new string which is 3 copies of the front.

front3('Java') → 'JavJavJav'
front3('Chocolate') → 'ChoChoCho'
front3('abc') → 'abcabcabc'

Go

...Save, Compile, Run (ctrl-enter)

Show Solution

```
def front3(str):  
    front = str[:3]  
    return front + front + front
```

Expected

Run

front3('Java') → 'JavJavJav'	'JavJavJav'	OK	
front3('Chocolate') → 'ChoChoCho'	'ChoChoCho'	OK	
front3('abc') → 'abcabcabc'	'abcabcabc'	OK	
front3('abcXYZ') → 'abcabcabc'	'abcabcabc'	OK	
front3('ab') → 'ababab'	'ababab'	OK	
front3('a') → 'aaa'	'aaa'	OK	
front3('') → ''	''	OK	



All Correct

next | [chance](#)

[Python](#) > [Warmup-1](#)

federror.tim@gmail.com [done page](#)

Your [progress graph](#) for this problem

Warmup-1 ★★★★★

chance

Simple warmup problems to get started, no loops (solutions available)

- | | | |
|------------------|--------------------|----------------|
| ✓ sleep_in H | ✓ monkey_trouble H | ✓ sum_double H |
| ✓ diff21 H | ✓ parrot_trouble H | ✓ makes10 H |
| ✓ near_hundred H | ✓ pos_neg H | ✓ not_string H |
| ✓ missing_char H | ✓ front_back H | ✓ front3 H |